智慧工廠設備異常警示 Agent 開發報告

Pegatron ML Engineer Assignment 3 ｜ 作者：郭逸凡 ｜ 語言：繁體中文

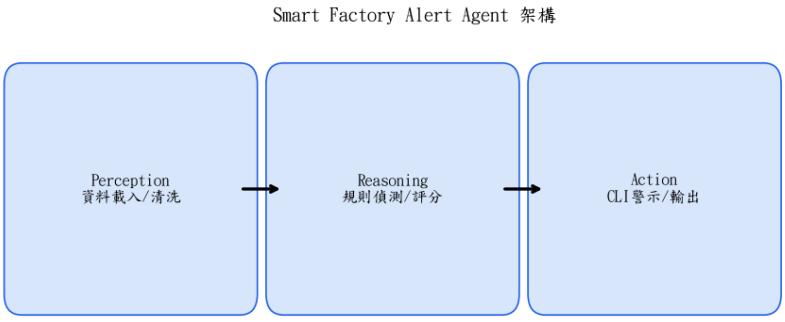
一、任務描述

本作業模擬智慧工廠設備感測器資料（溫度 temp、壓力 pressure、振動 vibration），建置一套設備異常警示 AI Agent，自動偵測模擬異常並輸出即時警示。資料集共 300 筆，每 5 分鐘一筆，約 10% 異常、5% 缺值，供清洗與偵測流程展示。

二、整體流程（Workflow）

1. 資料生成：generate_data.py 依固定 seed 產生 300 筆 CSV（含異常與缺值）。
2. 資料載入與驗證：preprocessing.load_data 驗證 schema、解析 timestamp。
3. 資料清洗：中位數補值、依 timestamp 排序、計算 z-score 標準化欄位。
4. 異常偵測：detector 以閾值規則判定異常，計算 0-100 分數、嚴重度與建議。
5. 警示輸出：Agent 於命令列列出異常、分數、建議，並寫入 alerts.csv / run_log.txt。

三、系統架構



Agent 採 Perception-Reasoning-Action 三層設計：Perception 負責資料載入與清洗；Reasoning 以規則引擎判定異常並評分；Action 輸出警示並存檔。工具以 registry 形式註冊（load_data / clean_data / detect / save_results），各步驟可獨立測試與替換。

四、模型與偵測細節

偵測採規則式閾值（可解釋、無外部依賴），閾值依題目規範：
- 溫度 temp：正常 45–50°C，異常 >52 或 <43
- 壓力 pressure：正常 1.00–1.05，異常 >1.08 或 <0.97
- 振動 vibration：正常 0.02–0.04，異常 >0.07

任一感測器超閾值即判定異常。分數 = 40×(異常感測器數) + 60×(最大偏離幅度)，上限 100。
偏離幅度 = 超出閾值的量 ÷（2×正常區間寬度），上限 1；即 temp ÷10、pressure ÷0.10、vibration ÷0.04，使不同量級的感測器分數可比較（temp 超出 5°C 與 vibration 超出 0.02 同分）。
嚴重度：多感測器或分數 >= 80 為 high；>= 50 為 medium；其餘 low。
另輸出可行動建議（例如檢查冷卻系統、軸承、泵浦等）。

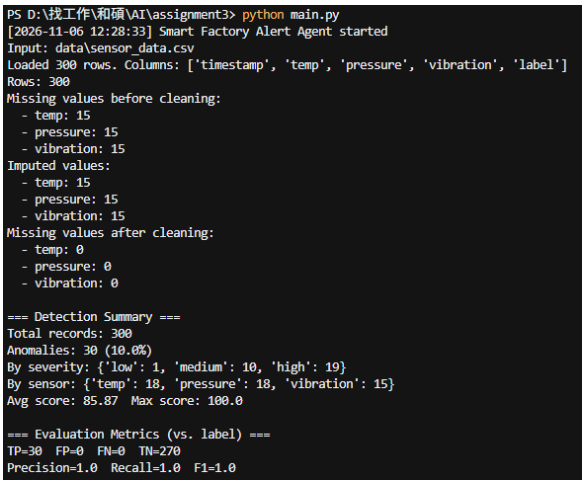
五、資料與清洗說明

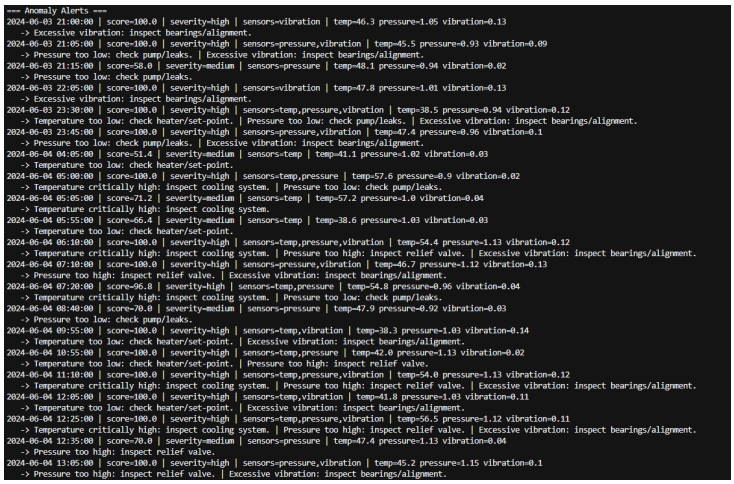
資料：300 筆、5 分鐘間隔、約 10% 異常、每欄約 5% 缺值。
清洗：中位數補值、timestamp 排序、z-score 標準化（產生 *_z 標準化欄位）。
測試：unittest 19 項全數通過（前處理、偵測邊界與評分、Agent 端到端與串流模式）。

指標

評估指標（以 label=abnormal 為 ground truth）：
TP=30 FP=0 FN=0 TN=270
Precision=1.0 Recall=1.0 F1=1.0
偵測異常數=30 (10.0%)，平均分數=85.87；嚴重度 high 19 / medium 10 / low 1

六、Demo 截圖



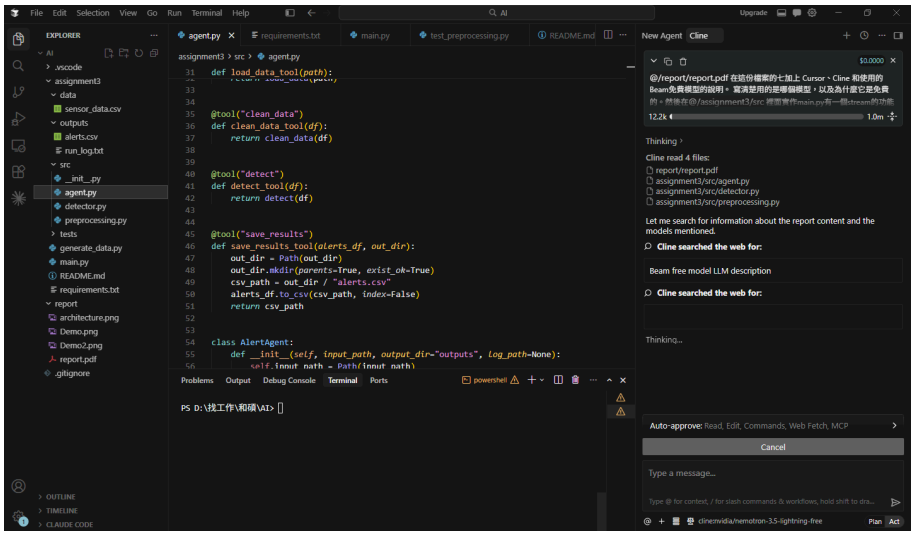


圖（左）：資料載入、缺值清洗、偵測摘要與評估指標；圖（右）：異常警示輸出（時間、分數、嚴重度、感測器與建議）。

七、AI 工具使用說明

使用工具與模型（皆為免費）：

- Cursor：以 VS Code 為基礎的 AI 程式編輯器，本專案以其免費方案作為開發環境。
- Cline：開源且免費的 AI 編碼 Agent 擴充套件，可讀寫檔案、執行終端機指令與搜尋網頁，並提供 Plan（先規劃）與 Act（再執行）兩種模式。Cline 本身不收費，費用只取決於所接的模型。
- 模型：NVIDIA Nemotron（nemotron-3.5-lightning），NVIDIA 推出的大型語言模型系列。本專案透過 Cline 內建的模型服務使用其免費版本 nvidia/nemotron-3.5-lightning-free（模型名稱即標示 free），使用時不需付費，介面上的費用計數顯示 $0.0000（見下圖右上角與右下角）。



圖：在 Cursor 中以 Cline Agent 開發本專案（右下角為使用模型，右上角為費用 $0.0000）

開發流程如下：

1. 需求確認（Plan Mode）：先以 Cline 的 Plan Mode 將題目需求與我對題目的理解交給 Agent 確認，並產出開發規格（spec）。過程中確認在「僅能使用免費 AI 工具」且「題目已明確給定異常閾值」的情境下，純邏輯規則判斷的 Agent 為最佳解：判斷依據透明可解釋、結果穩定可重現、不需額外訓練資料，也不依賴付費服務。
2. 平行開發：規格確認後，請 Cline 撰寫資料產生專用的 prompt。由於資料產生與程式實作彼此獨立，我另開一個 Cline session 負責資料產生，原 session 同步進行程式實作。程式實作耗時較長，完成後自動銜接已產生的資料，接著撰寫單元測試並一併執行驗證。
3. 文件與報告：最後由 Agent 產出報告初稿（含架構圖）與 README。因一開始即要求 README 列出精簡的專案結構並註明各檔案用途，人工 code review 得以快速完成；實際執行程式後也確認結果正確。
4. 人工修訂：我只需補充想表達的內容，並修訂報告與 README，即完成全部交付物。

八、使用方式

環境需求：Python >= 3.9；依賴 pandas>=2.0.0、numpy>=1.24.0（見 requirements.txt）
1. 安裝：pip install -r requirements.txt
2. 產生資料：python generate_data.py（輸出 data/sensor_data.csv）
3. 執行 Agent：python main.py（可加 --input / --output；加 --stream 逐筆串流輸出警示）
4. 執行測試：python -m unittest discover -s tests

輸出：outputs/alerts.csv（異常明細）、outputs/run_log.txt（執行記錄與 Precision/Recall/F1）

九、結論

本專案完成資料生成、清洗、規則式異常偵測、CLI 警示輸出、Demo 截圖與技術報告，
並以 19 項單元測試確保結果正確。

十、程式碼

GitHub：https://github.com/evankuo2017/PEGAAi_ML_Engineer_Assignment3_evankuo2017